

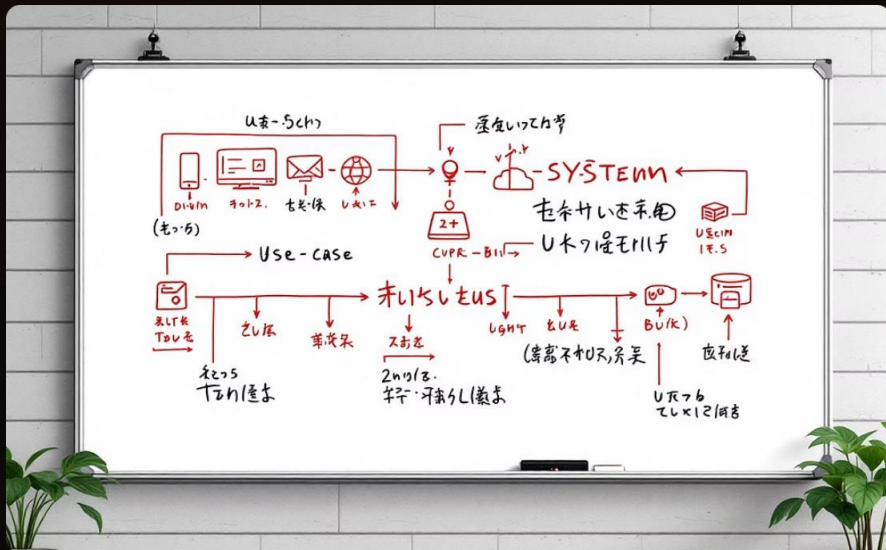


Sistema de Moeda Estudantil: Escolhas Técnicas do Lab03

Foco: demonstrar que a Release 1 (Lab03) entregou a modelagem, a arquitetura e os CRUDs iniciais utilizando tecnologias simples e familiares, Python, Django e HTML básico, priorizando objetividade, portabilidade e alinhamento acadêmico.

R1: Base e Modelagem (Lab03S01 e S02)

A entrega de modelagem inclui diagramas UML de Casos de Uso, Classes e Componentes, além do Modelo Entidade-Relacionamento (ER). A modelagem priorizou clareza e mapeamento direto para os modelos Django, facilitando a transformação de entidades em classes Python persistidas via ORM.



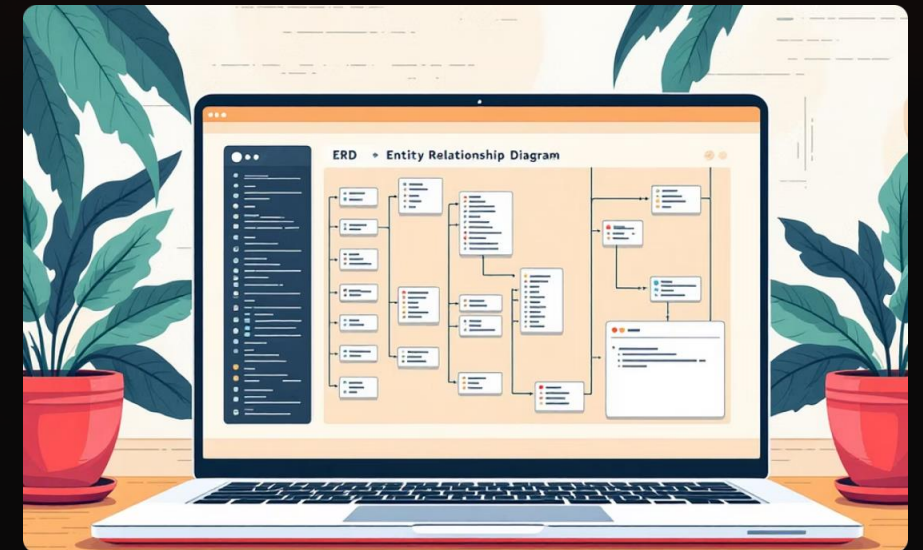
Diagramas de Caso de Uso

Mapeamento das interações principais: aluno, professor e parceiro com o sistema de moeda.



Diagrama de Classes

Entidades principais (Aluno, Professor, Parceiro, Conta, Transação) com atributos essenciais e relacionamentos.



Modelo ER

Chaves e cardinalidades definidas para consistência referencial; modelo pronto para migração Django.

Backend: Python e Django – Familiaridade e Objetivo

Decisão técnica: usar Python e Django por serem amplamente conhecidos na disciplina e permitirem desenvolvimento rápido sem curva íngreme. Django fornece estrutura MVT (equivalente a MVC) que separa apresentação, controle e modelos, compatível com exigências acadêmicas e manutenção futura.

Vantagens

- Produtividade: scaffolding rápido e conventions-over-configuration.
- Comunidade e documentação: recursos abundantes para suporte acadêmico.
- Testabilidade: facilita validação de requisitos e automação de testes.

Front-end

HTML básico, CSS e JavaScript mínimo; foco em formulários e templates Django para entregar protótipos funcionais sem overhead tecnológico.



Justificativa Técnica: ORM e Simplicidade

A estratégia técnica priorizou abstração e simplicidade: usar o ORM do Django elimina a necessidade de escrever SQL direto, permitindo concentrar esforços na lógica de negócio e validações. Para R1 escolhemos SQLite por portabilidade, zero-config e compatibilidade com a avaliação em ambiente acadêmico.

1

ORM (Django Models)

Mapeamento direto: classes Python representam tabelas, facilitando migrações e testes. Reduzam erros de consulta manual.

2

SQLite

Banco leve, arquivo único, ideal para entrega de laboratório e avaliação local, facilita réplica por avaliadores e professores.

3

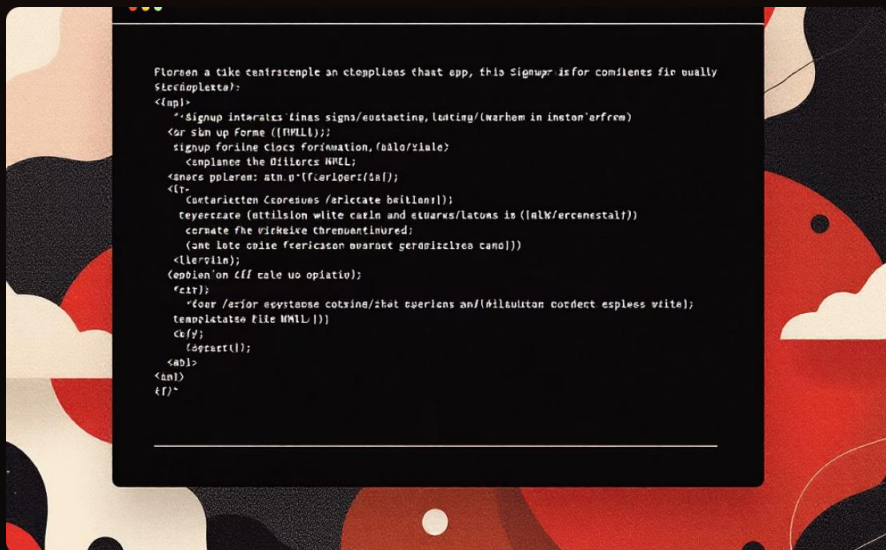
Admin Panel

Interface administrativa pronta do Django para gerenciar pré-cadastros, acelerar testes e demonstrar funcionalidades sem UI completa.



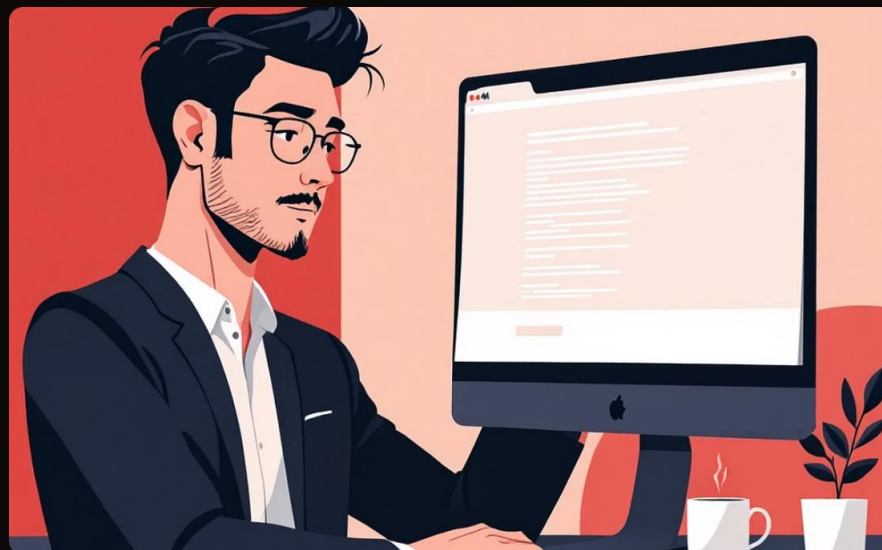
Implementação da R1: CRUD Aluno e Parceiro

Meta da Release 1: implementar CRUDs essenciais para Aluno e Empresa Parceira (criar e visualizar registros). Os templates e rotas foram nomeados de forma descritiva para facilitar avaliação e teste.



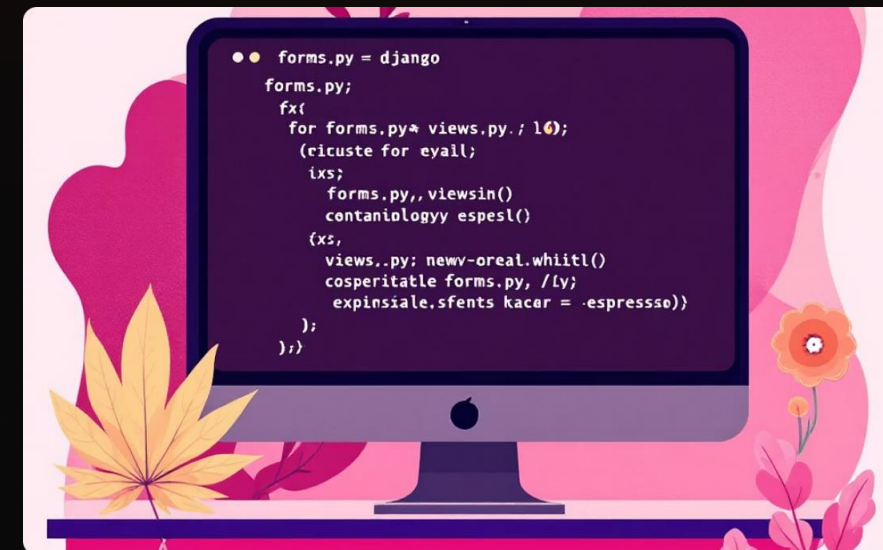
Aluno

Template: `signup_aluno.html`, formulário com validação básica; dados persistidos via Student model e `forms.py`; view associada registra e redireciona para listagem.



Parceiro

Template: `empresa_create.html`, cadastro de parceiros com campos de contato e categoria; view e form específicos garantem validação e criação segura.



Camadas

Validação em `forms.py`; lógica de solicitação em `views.py`; modelos centralizam regras de integridade (unique constraints, choices).



Revisão: Alinhamento Código-Modelo

A arquitetura MVT implementada segue fielmente os diagramas entregues: nomes de classes, atributos e relacionamentos espelham o ER. O uso do Django Auth e do Admin garante controle de acesso e administração na entrega, atendendo requisitos de segurança acadêmica mínimos.

1 Consistência

Modelos Python correspondem diretamente às entidades do diagrama, facilitando validação e revisão por avaliadores.

2 Segurança

Autenticação básica via Django Auth para acessar áreas administrativas e endpoints de teste.

3 Status da R1

Concluída: modelagem, migrações, views básicas, templates de criação/visualização e painel administrativo operacional.



R1 Concluída: Base Sólida e Objetiva

Conclusão: as decisões técnicas de Python, Django e HTML básico, priorizaram familiaridade e entrega objetiva, permitindo cumprir o escopo do Lab03 com qualidade. A base resultante é portátil, auditável e pronta para evoluir.

Próximos Passos (R2)

1. Implementar transações e lógica de saldo (modelo Conta/Transação).
2. Extrato do usuário com paginação e filtros.
3. Catálogo de vantagens e integração com Parceiros.

Observação Final

Para avaliação técnica: disponibilizo repositório com README, instruções de setup (virtualenv, migrate, runserver) e scripts de carga inicial para facilitar revisão.